ARIN5305 – Artificial Intelligence in Software Engineering

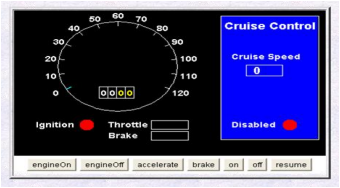
Reviews

Shing-Chi Cheung
Department of Computer Science & Engineering
HKUST

Course Review

Here! Try it out Yourself!

(http://www.cse.ust.hk/~scc/teaching/CruiseControl.html)



Recall: Static and Dynamic Analysis

- Static Analysis: Validate without program execution
  - This include code review and symbolic execution
- Dynamic Analysis: Validate by executing the program with real inputs
  - Testing
- Big Code Analysis: Validate based on big data mining from public repositories and forums

Software Faults, Errors & Failures

- Software Fault : A static defect in the software

Faults in software are design mistakes and will always exist

- Software Error : An incorrect internal state that is the manifestation of some fault
- Software Failure : External, incorrect behavior with respect to the requirements or other description of the expected behavior

Illustrative Example

```
public static int numZero (int[] x) {
// Effects: if x == null throw NullPointerException
// else return the number of occurrences of 0 in x
  int count = 0;
  for (int i = 1; i < x.length; i++) {
      if (x[i] == 0) {
            count++;
      }
  }
  return count;
}
```


Illustrative Example – Software Fault

```
public static int numZero (int[] x) {
// Effects: if x == null throw NullPointerException
// else return the number of occurrences of 0 in x
  int count = 0;
  for (int i = 1; i < x.length; i++) {
      if (x[i] == 0) {
            count++;
      }
  }
  return count;
}
```


Illustrative Example – Software Error

```
public static int numZero (int[] x) {
// Effects: if x == null throw NullPointerException
// else return the number of occurrences of 0 in x
  int count = 0;
  for (int i = 1; i < x.length; i++) {
      if (x[i] == 0) {
            count++;
      }
  }
  return count;
}
```

numZero([2, 7, 0])

Any failure occur?
Any error occur?

Illustrative Example – Software Error

```
public static int numZero (int[] x) {
// Effects: if x == null throw NullPointerException
// else return the number of occurrences of 0 in x
  int count = 0;
  for (int i = 1; i < x.length; i++) {
      if (x[i] == 0) {
            count++;
      }
  }
  return count;
}
```

numZero([2, 7, 0])

Any failure occur?
Any error occur?

State error occurs at the first iteration where the state is (x=[2,7,0], count=0, i=1, PC=if). The correct state should be (x=[2,7,0], count=0, i=0, PC=if).

Illustrative Example – Software Error

```
public static int numZero (int[] x) {
// Effects: if x == null throw NullPointerException
// else return the number of occurrences of 0 in x
  int count = 0;
  for (int i = 1; i < x.length; i++) {
      if (x[i] == 0) {
            count++;
      }
  }
  return count;
}
```

numZero([0, 7, 2])

Any failure occur?
Any error occur?

Can you think of a program example and a test case that triggers a fault but causes no error and failure?

```
int square (int x) {
  int result = x + x;
  return result;
}
```

Test case: square(2)

Testing & Debugging

- Testing : Finding inputs that cause the software to fail
- Debugging : The process of finding a fault given a failure

Fault & Failure Model

Three conditions necessary for a failure to occur

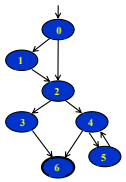
1. Reachability : The location or locations in the program that contain the fault must be reached
2. Infection : The state of the program must be incorrect
3. Propagation : The infected state must propagate to cause some output of the program to be incorrect

Fault & Failure Model

Three conditions necessary for a failure to occur

1. Reachability : The location or locations in the program that contain the fault must be reached
2. Infection : The state of the program must be incorrect
3. Propagation : The infected state must propagate to cause some output of the program to be incorrect

Structural Coverage Example



Node Coverage
TR = { 0, 1, 2, 3, 4, 5, 6 }
Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 5, 4, 6]

Edge Coverage
TR = { (0,1), (0,2), (1,2), (2,3), (2,4), (3,6), (4,5), (4,6), (5,4) }
Test Paths: [0, 1, 2, 3, 6] [0, 2, 4, 5, 4, 6]

Edge-Pair Coverage
TR = { [0,1,2], [0,2,3], [0,2,4], [1,2,3], [1,2,4], [2,3,6], [2,4,5], [2,4,6], [4,5,4], [5,4,5], [5,4,6] }
Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 6] [0, 2, 3, 6] [0, 2, 4, 5, 4, 5, 4, 6]

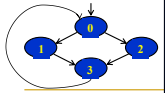
Complete Path Coverage
Test Paths: [0, 1, 2, 3, 6] [0, 1, 2, 4, 6] [0, 1, 2, 4, 5, 4, 6] [0, 1, 2, 4, 5, 4, 5, 4, 6] [0, 1, 2, 4, 5, 4, 5, 4, 5, 4, 6] ...

Simple Paths and Prime Paths

- Simple Path : *A path from node n_i to n_j is simple if no node appears more than once, except possibly the first and last nodes are the same*
  - No internal loops
  - May include other subpaths
  - A loop is a simple path

Any paths can be created by composing simple paths!

- Prime Path : *A simple path that does not appear as a proper subpath of any other simple path*

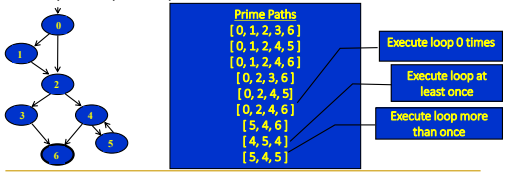


Simple Paths : [0, 1, 3, 0], [0, 2, 3, 0], [1, 3, 0, 1], [2, 3, 0, 2], [3, 0, 1, 3], [3, 0, 2, 3], [1, 3, 0, 2], [2, 3, 0, 1], [0, 1, 3], [0, 2, 3], [1, 3, 0], [2, 3, 0], [3, 0, 1], [3, 0, 2], [0, 1], [0, 2], [1, 3], [2, 3], [3, 0], [0], [1], [2], [3]

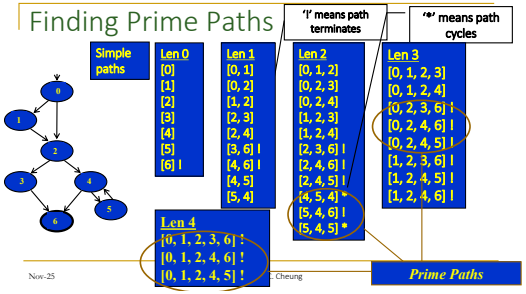
Prime Paths : [0, 1, 3, 0], [0, 2, 3, 0], [1, 3, 0, 1], [2, 3, 0, 2], [3, 0, 1, 3], [3, 0, 2, 3], [1, 3, 0, 2], [2, 3, 0, 1]

More Prime Path Example

- The following graph has 38 simple paths
- Only nine *prime paths*



Finding Prime Paths



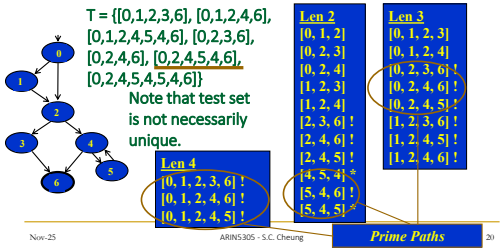
Prime Path Coverage

- A simple, elegant and finite criterion that requires loops to be executed as well as skipped

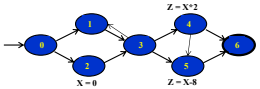
Prime Path Coverage (PPC) : TR contains each prime path in G.

- **Will tour all paths of length 0, 1, …**
- **That is, it subsumes node, edge, and edge-pair coverage**

Test Set Construction for Prime Path Coverage

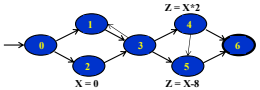


Data Flow Testing Exercise



All-defs for *X*

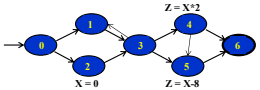
Data Flow Testing Exercise



All-defs for *X*
[2, 3, 4]
or
[2, 3, 5]
or
[2, 3, 4, 5]

All-uses for *X*

Data Flow Testing Exercise

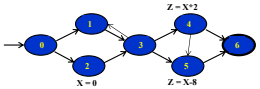


All-defs for *X*
[2, 3, 4]
or
[2, 3, 5]
or
[2, 3, 4, 5]

All-uses for *X*
[2, 3, 4]
[2, 3, 5]
or
[2, 3, 4, 5]

All-du-paths for *X*

Data Flow Testing Exercise

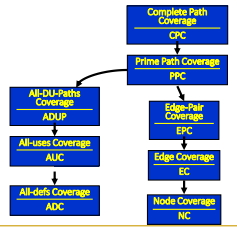


All-defs for *X*
[2, 3, 4]
or
[2, 3, 5]
or
[2, 3, 4, 5]

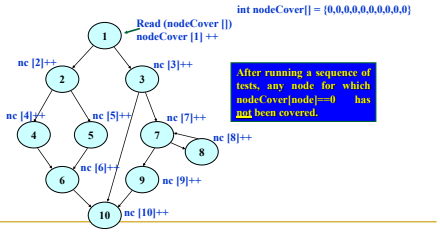
All-uses for *X*
[2, 3, 4]
[2, 3, 5]
or
[2, 3, 4, 5]

All-du-paths for *X*
~~[2, 3, 4]~~
[2, 3, 5]
[2, 3, 4, 5]

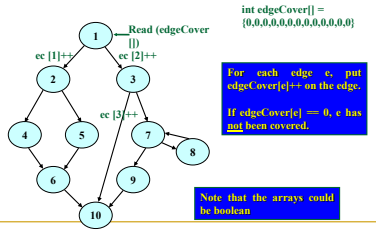
Graph Coverage Criteria Subsumption



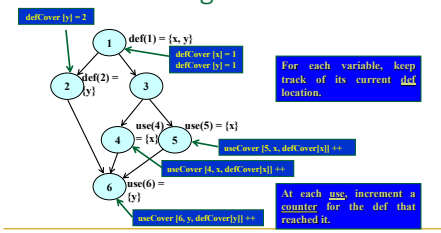
Statement Coverage Example



Edge Coverage Instrumentation

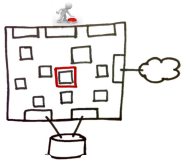


All-Uses Coverage Instrumentation



What is Unit Testing?

- To isolate each part of program and show that individual parts are correct.
- JUnit is a de facto framework used by Java practitioners to test their code.
- It is a built-in Eclipse plug-in.
- Its latest version is 5.



Putting it in Eclipse

```
public class RectangleTest {

 Rectangle r = new Rectangle(2, 4);
 final int height = 2, width = 4;
 final double tolerance = 0.000001;

@Test
public void testGetArea() {
  double expectedArea = width * height;
  double actualArea = r.getArea();
  assertEquals(expectedArea, actualArea,
      tolerance, "getArea fails");
}
```

```
@Test
public void testResize() {
  r.resize(0.25, 2);
  assertEquals(0.25*height, r.getHeight(), tolerance,
      "resize - height fails");
  assertEquals(2*width, r.getWidth(), tolerance,
      "resize - width fails");
}
... // other tests
}
```


BeforeEach and AfterEach test case

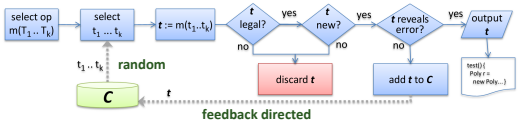
```
@BeforeEach
public void setUp() throws Exception {
        // create objects and common variables for your tests here
        // setUp() is called before EACH test case is run
}

@AfterEach
public void tearDown() throws Exception {
        // put code here to reset or release objects, e.g., p1=null;
        // to garbage collect unused objects or resources
        // tearDown() is called after EACH test case is run
}
```


- No need to write tests
- No need to locate faults
- No need to fix faults

Grand Challenge

Directed Random Test Generator



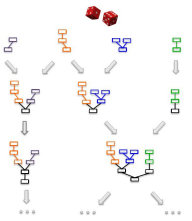
- *C* is a repository containing possible terms used by the Test Generator.

Directed Random Test Generator

- Generate simple inputs randomly from repository *C*.

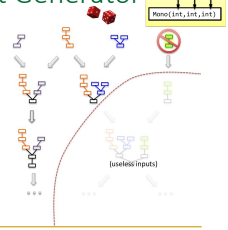
Directed Random Test Generator

- Generate simple inputs randomly from repository *C*.
- Build new inputs incrementally from previous ones.



Directed Random Test Generator

- Executes inputs
- Discards the ones useless for extension
  - illegal, redundant
- Prune input space



Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

[a>b] x=(a+b)-a = b

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

[a>b] x=(a+b)-a = b

[a>b] b-a >? 0

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

[a>b] x=(a+b)-a = b

[a>b] b-a >? 0

[a>b, b-a<=0] END

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

[a>b] x=(a+b)-a = b

[a>b] b-a >? 0

[a>b, b-a<=0] END

[a>b, b-a>0] fail

Symbolic Testing (a.k.a. Symbolic Execution)

```
foo (int x, int y) {
  if (x>y) {
    x = x + y;
    y = x - y;
    x = x - y;
    if (x - y > 0) {
      assert (false); // bug
    }
  }
}
```

x=a, y=b

a <? b

[a<=b] END

[a>b] x=a+b

[a>b] y=(a+b)-b = a

[a>b] x=(a+b)-a = b

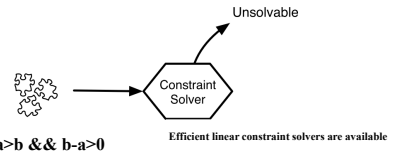
[a>b] b-a >? 0

[a>b, b-a<=0] END

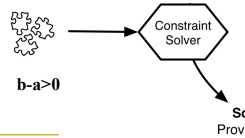
[a>b, b-a>0] fail

Constraints:
a>b && b-a>0

Using a Linear Constraint Solver



Constraint Solving with What-if Analysis



Concolic = Concrete + Symbolic

```
int foo(int x, int y) {
 int z = square(x);
 if (z > 100 && y > 20)
  assert(false);
 return y*z;
}
```

x = 15, y = 10 → z = 225 → 225 > 100 && 10 > 20 → return 2250

Execute program concretely

Test: foo(15, 10)

Concolic = Concrete + Symbolic

```
int foo(int x, int y) {
 int z = square(x);
 if (z > 100 && y > 20)
  assert(false);
 return y*z;
}
```

x = 15, y = 10 → z = 225 → 225 > 100 && 10 > 20 → return 2250

x = X, y = Y → z = square(X) → ?(square(X) > 100 && Y > 20) → [!(square(X) > 100 && Y > 20)] return Y*square(X)

Execute program concretely
Collect symbolic path condition

Test: foo(15, 10)

Concolic Testing

```
int foo(int x, int y) {
 int z = square(x);
 if (z > 100 && y > 20)
  assert(false);
 return y*z;
}
```

x = 15, y = 10 → z = 225 → 225 > 100 && 10 > 20 → return 2250

x = X, y = Y → z = square(X) → ?(square(X) > 100 && Y > 20) → [square(X) > 100, Y > 20] / [!(square(X) > 100 && Y > 20)] return Y*square(X)

Execute program concretely
Collect symbolic path condition
Negate a constraint on the path condition and solve it

Test: foo(15, 10)

Concolic Testing

```
int foo(int x, int y) {
 int z = square(x);
 if (z > 100 && y > 20)
  assert(false);
 return y*z;
}
```

x = 15, y = 10 → z = 225 → 225 > 100 && 10 > 20 → return 2250

x = X, y = Y → z = square(X) → ?(square(X) > 100 && Y > 20) → [225 > 100, Y > 20] / [!(square(X) > 100 && Y > 20)] return Y*square(X)

assert(false)

Test: foo(15, 10)

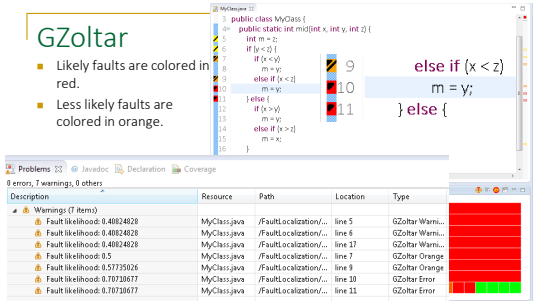
Fault Localization

mid( ) {	Runs					
int x, y, z, m;	1	2	3	4	5	6
read("Enter 3 numbers:", x, y, z);						
m = z;						
if (y < z) {						
if (x < y)						
m = y;						
else if (x < z)						
m = y;						
} else {						
if (x > y)						
m = y;						
else if (x > z)						
m = x;						
}						
print("Middle number is:", m);						
}	✓	✓	✓	✓	×	✓

- Runs
  - 1: (1,1,2)
  - 2: (0,1,2)
  - 3: (2,1,0)
  - 4: (0,2,1)
  - 5: (1,0,2)
  - 6: (2,0,1)

GZoltar

- Likely faults are colored in red.
- Less likely faults are colored in orange.



Fault Localization

mid( ) {	Runs					
int x, y, z, m;	1	2	3	4	5	6
read("Enter 3 numbers:", x, y, z);	•	•	•	•	•	•
m = z;	•	•	•	•	•	•
if (y < z) {	•	•	•	•	•	•
if (x < y)	•	•			•	•
m = y;		•				
else if (x < z)	•				•	•
m = y;	•				•	
} else {			•	•		
if (x > y)			•	•		
m = y;			•			
else if (x > z)				•		
m = x;						
}						
print("Middle number is:", m);	•	•	•	•	•	•
}	✓	✓	✓	✓	×	✓

- Runs
  - 1: (1,1,2)
  - 2: (0,1,2)
  - 3: (2,1,0)
  - 4: (0,2,1)
  - 5: (1,0,2)
  - 6: (2,0,1)

Fault Localization

mid( ) {	Runs					
int x, y, z, m;	1	2	3	4	5	6
read("Enter 3 numbers:", x, y, z);	•	•	•	•	•	•
m = z;	•	•	•	•	•	•
if (y < z) {	•	•	•	•	•	•
if (x < y)	•	•			•	•
m = y;		•				
else if (x < z)	•				•	•
m = y; // *** BUG ***	•				•	
} else {			•	•		
if (x > y)			•	•		
m = y;			•			
else if (x > z)				•		
m = x;						
}						
print("Middle number is:", m);	•	•	•	•	•	•
}	✓	✓	✓	✓	×	✓

- Runs
  - 1: (1,1,2)
  - 2: (0,1,2)
  - 3: (2,1,0)
  - 4: (0,2,1)
  - 5: (1,0,2)
  - 6: (2,0,1)

Fault Localization

mid( ) {	Runs					
int x, y, z, m;	1	2	3	4	5	6
read("Enter 3 numbers:", x, y, z);	•	•	•	•	•	•
m = z;	•	•	•	•	•	•
if (y < z) {	•	•	•	•	•	•
if (x < y)	•	•			•	•
m = y;		•				
else if (x < z)	•				•	•
m = y; // *** BUG ***	•				•	
} else {			•	•		
if (x > y)			•	•		
m = y;			•			
else if (x > z)				•		
m = x;						
}						
print("Middle number is:", m);	•	•	•	•	•	•
}	✓	✓	✓	✓	×	✓

- Premise
  - Bug participates more often in failing runs than successful runs
  - RIP model
    - Reachability
    - Infection
    - Propagation

Ranking function - Tarantula

- J. A. Jones and M. J. Harrold, "Empirical evaluation of the Tarantula automatic fault-localization technique," in *Proc. of the 20th IEEE/ACM Conference on Automated Software Engineering*, pp. 273-282, Long Beach, California, USA, December, 2005

$$X/X+Y,\ X=(N_{EF}/N_F)\ \&\ Y=(N_{ES}/N_S)$$

N_{EF} : Number of failing runs executing the statement

N_{ES} : Number of successful runs executing the statement

N_F : Number of failing runs

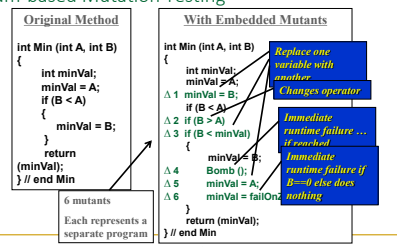
N_S : Number of successful runs

Fault Localization

X/X+Y, X=(N_{EF}/N_F) & Y=(N_{ES}/N_S)
N_F: 1, N_S: 5

mid( ) {	Runs						Tarantula
int x, y, z, m;	1	2	3	4	5	6	
read("Enter 3 numbers:", x, y, z);	•	•	•	•	•	•	0.5
m = z;	•	•	•	•	•	•	0.5
if (y < z) {	•	•	•	•	•	•	0.5
if (x < y)	•	•			•	•	0.625
m = y;		•					0.0
else if (x < z)	•				•	•	0.714
m = y; // *** BUG ***	•				•		0.833
} else {			•	•			0.0
if (x > y)			•	•			0.0
m = y;			•				0.0
else if (x > z)				•			0.0
m = x;							0.0
}							0.0
print("Middle number is:", m);	•	•	•	•	•	•	0.5
}	✓	✓	✓	✓	×	✓	

Program-based Mutation Testing



Syntax-Based Coverage Criteria

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m.

- **The RIP model:**
  - **Reachability : The test causes the faulty statement to be reached (in mutation – the mutated statement)**
  - **Infection : The test causes the faulty statement to result in an incorrect state**
  - **Propagation : The incorrect state propagates to incorrect output**
- **The RIP model leads to two variants of mutation coverage …**

Syntax-Based Coverage Criteria

- **1) Strongly Killing Mutants:**

  Given a mutant $m \in M$ for a program P and a test t, t is said to strongly kill m if and only if the output of t on P is different from the output of t on m
- **2) Weakly Killing Mutants:**

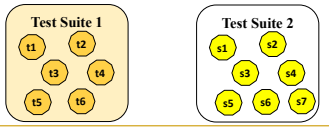
  Given a mutant $m \in M$ that modifies a location l in a program P, and a test t, t is said to weakly kill m if and only if the state of the execution of P on t is different from the state of the execution of m immediately on t after l
  - **Weakly killing satisfies reachability and infection, but not propagation**

Search-based Test Generation

- Given class under test (CUT), EvoSuite automatically generates a test suite using a genetic algorithm
  1. Create 2 initial test suites by adding method calls randomly and insert the test suites into current generation
  2. Select two test suites from current generation
  3. Create 2 new test suites by crossover (exchange test cases of the suites)
  4. Modify two test suites (test drivers) from step 3 with mutation operators (remove, insert, change operators)
  5. Insert the new two test suites from step 4 to next generation if coverage of the new two test suites are higher than that of parents
  6. Repeat 2~5 until next generation has sufficient number of test suites
  7. Repeat 2~6 until time limit is reached or all branches are covered
  8. Select a test suit with the highest branch coverage and insert assertions by observing differences in execution traces between the original CUT and a mutated CUT

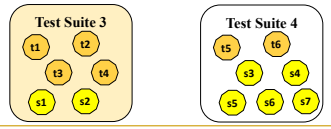
EvoSuite

1. Create 2 initial test suites by adding method calls randomly and insert the test suites into current generation

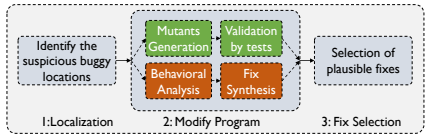


EvoSuite

2. Select two test suites from current generation
3. Create 2 new test suites by crossover (exchange test cases of the suites)



General Process

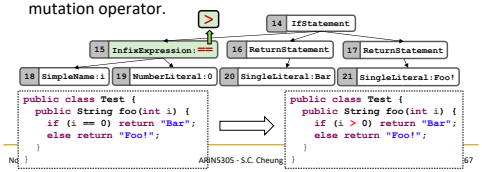


Mutation-driven (also known as generate-and-validate)

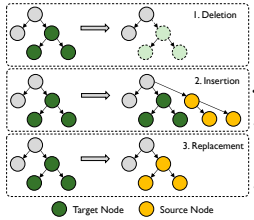
Semantics-driven

Mutation Operators

- A mutant is a new version of a program that is created by making a small syntactic change to the original program.
- An operation which creates a mutant of a program is called a mutation operator.



Mutation Operators

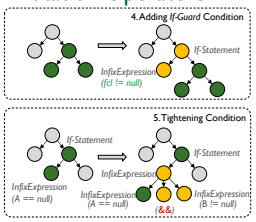


Redundancy Assumption
[Qi et al., ICSE14, Tao et al., FSE14]

The fix ingredients already exist elsewhere in the code.

- The same class *[Debroy et al., ICSTVV10, Martinez et al., ICSE14]*
- The same application *[LeGoues et al., ICSE12, Weimer et al., ASE13, DeMarco et al., ISSTA14 , Long et al., ESEC/FSE15, Long et al., POPL16]*
- Other applications *[Sidiroglou-Douskos et al., PLDI15]*

Mutation Operators



Target Node New Node

```
11      project.getProject();
12      ....
13 +    if (fcl != null) {
14        removeFileListener(fcl);
15 +    }
16
```

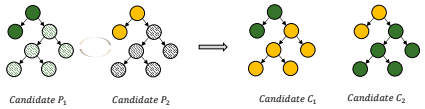
Many real bug fixes are related to buggy if conditions [EmSE 2009]

```
11 +  if (A == null && B != null){
12      result = annotated;
13    } else {
15      result = getResult();
16    }
```

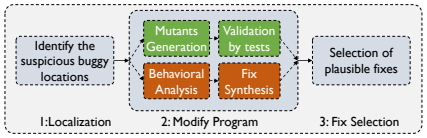
Loosening condition can be achieved in the same way by replacing the operator && with ||

Patch Generation - GenProg

- Mutation Operator: *Insertion*, *Replacement*, and *Deletion*.
- Crossover Operator: Cross over between two candidates.



General Process



Mutation-driven (also known as generate-and-validate)

Semantics-driven

Condition Fix Synthesis – SPR [ESEC/FSE15]

```
void foo(...) {
    ...
    char *buf = malloc(nsize);
    ...
    memcpy(buf, a, nsize);
    ...
}
```

Add a potentially guarded control statement

- Create a template fix with an *abstract condition*
- Try different values of the *abstract condition* during the symbolic execution (initially set to 0) Flip the value when the execution failed.
- Record the *variable values* at each condition invocation(*buf, nsize*)
- Synthesize a concrete fix using the recorded variables

Condition Fix Synthesis – SPR [ESEC/FSE15]

```
void foo(...) {
    ...
    char *buf = malloc(nsize);
    if (_abst_cond())
        return;
    memcpy(buf, a, nsize);
    ...
}
```

	_abst_cond()	buf	nsize	
1st	0	0x30aa...	100	
2nd	1	0	100000	F→P
3rd	0	0x30ab...	200	
4th	1	0	200000	F→P

SPR fixes **38** bugs out of **69**, while **GenProg** fixes only **16**, **AE** fixes **25**.

_abst_cond() ⟹ *buf == 0*

RHS of Assignment – SemFix [ICSE13]

- An *expression* is treated as a *function*
- The repair candidates are in one of the two forms:
  - $x = F_{buggy}(\ldots) \rightarrow x = F_{fixed}(\ldots)$
  - $if\ (F_{buggy}(\ldots)) \rightarrow if(F_{fixed}(\ldots))$
- The function takes all the *accessible variables* as parameters

RHS of Assignment – SemFix [ICSE13]

```
1  int G(int a, int b, int c) {
2      int bias;
3      if (a)
4          bias=c; bias = F(a,b,c)
5      else
6          bias = b;
7      if (bias > c)
8          return 1;
9      else return 0;
10 }
```

Test	Inputs a	b	c	Expected output	Observed output	Status
1	1	0	100	0	0	Pass
2	1	11	110	1	0	Fail
3	0	100	50	1	1	Pass
4	1	-20	60	1	0	Fail
5	0	0	10	0	0	Pass

Test 2 Test 1 Test 4

$< 1,11,110 >$ $< 1,0,100 >$ $< 1,-20,60 >$

$F(a,b,c) = b + 100$ ⟸ $bias = F(1,11,110) > 110 \wedge F(1,0,100) \leq 100 \wedge F(1,-20,60) > 60$

End of Review

Examination will be open notes!